# REDIS

## From Scratch

Complete Zero-to-Hero Build Guide

Git • C/C++ • Networking • RESP Protocol • Data Structures
Persistence • Replication • Interview Prep

**This document covers EVERYTHING you need to build a production-grade Redis clone — even from zero.**

CodeCrafters Project Ready • Interview Ready

Build Your Own Redis • May 24, 2026

# Contents

# 1 Environment Setup — Before You Write a Single Line

## 1.1 What You Need

| Tool | Why | Check Version |
|---|---|---|
| GCC / G++ 11+ | Compile C/C++ code | `gcc -version` |
| Make | Build automation | `make -version` |
| Git | Version control | `git -version` |
| valgrind | Memory leak detection | `valgrind -version` |
| netcat (nc) | Test TCP connections | `nc -version` |
| redis-cli | Test against real Redis | `redis-cli -version` |

## 1.2 Install on Ubuntu/Debian

install_deps.sh

```
# Update package list
sudo apt-get update

# Install build tools
sudo apt-get install -y build-essential gcc g++ make

# Install Git
sudo apt-get install -y git

# Install debugging tools
sudo apt-get install -y valgrind gdb

# Install netcat and curl
sudo apt-get install -y netcat-openbsd curl

# Install real Redis (for reference / testing)
sudo apt-get install -y redis-server redis-tools

# Verify everything
gcc --version && git --version && redis-cli --version
```

## 1.3 Install on macOS

macOS Setup

```
# Install Homebrew if not present
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/
    HEAD/install.sh)"

# Install tools
brew install gcc make git redis netcat
```

# 2 Git — Complete Crash Course

NOTE

Git is a **distributed version control system**. Every change you make is tracked. You need it to submit CodeCrafters stages and collaborate on code.

## 2.1 Core Concepts

| Concept | What it means |
|---|---|
| **Repository (repo)** | A folder tracked by Git |
| **Commit** | A saved snapshot of your code |
| **Branch** | An independent line of development |
| **Remote** | A copy of the repo on a server (GitHub) |
| **Working directory** | Files you're currently editing |
| **Staging area (index)** | Files ready to be committed |
| **HEAD** | Pointer to your current commit |

## 2.2 First-Time Git Setup

git_first_setup.sh

```
# Tell Git who you are (run once per machine)
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"

# Set default branch name to main
git config --global init.defaultBranch main

# Set VS Code as the default editor
git config --global core.editor "code --wait"

# Verify your config
git config --list
```

## 2.3 Starting a Project

git_start.sh

```
# Option A: Start from scratch
mkdir my-redis && cd my-redis
git init                              # Creates .git folder

# Option B: Clone CodeCrafters repo (they provide this URL)
git clone https://github.com/codecrafters-io/build-your-own-redis
cd build-your-own-redis
```

## 2.4 The Daily Git Workflow

**The 3-Zone Model of Git:**

Working Directory $\xrightarrow{\text{git add}}$ Staging Area $\xrightarrow{\text{git commit}}$ Repository $\xrightarrow{\text{git push}}$ Remote

daily_workflow.sh

```
# 1. See what changed
git status

# 2. See exact differences
git diff                              # unstaged changes
git diff --staged                     # staged changes

# 3. Stage files
git add server.c                      # specific file
git add src/                          # entire folder
```

```
git add .                          # everything in current dir

# 4. Commit with a message
git commit -m "feat: implement PING command"

# 5. Push to GitHub
git push origin main

# 6. Pull latest changes
git pull origin main
```

## 2.5 Branches

git_branches.sh

```
# Create and switch to a new branch
git checkout -b feature/ping-command

# List all branches
git branch -a

# Switch to an existing branch
git checkout main

# Merge a branch into current branch
git merge feature/ping-command

# Delete a branch (after merging)
git branch -d feature/ping-command

# Push branch to remote
git push origin feature/ping-command
```

## 2.6 Undoing Mistakes

git_undo.sh

```
# Unstage a file (keep changes in working dir)
git restore --staged server.c

# Discard changes in working directory (DANGEROUS - permanent)
git restore server.c

# Undo last commit but keep changes staged
git reset --soft HEAD~1

# Undo last commit and unstage changes
git reset --mixed HEAD~1

# Completely undo last commit (DANGEROUS - loses changes)
git reset --hard HEAD~1

# See commit history
git log --oneline --graph --all
```

### 2.7 .gitignore

.gitignore

```
# Compiled binaries
*.o
*.out
server
my_redis

# Build directories
build/
bin/

# Editor files
.vscode/
*.swp
*~

# OS files
.DS_Store
Thumbs.db
```

**INTERVIEW Q&A**

**Q: What is the difference between `git merge` and `git rebase`?**

**A:** `merge` creates a new merge commit combining two branches — history is preserved. `rebase` rewrites your branch's commits on top of another branch — history is linear but commit hashes change. Use merge for public branches; use rebase to clean up local work before sharing.

**Q: What does HEAD mean?**

**A:** HEAD is a pointer to the current commit (or branch tip) you are working from. When you commit, HEAD moves forward. `HEAD~1` means one commit behind current HEAD.

## 3 C/C++ Fundamentals for Redis

### 3.1 Why C for Redis?

Real Redis is written in C because: (1) direct memory management, (2) no GC pauses, (3) predictable latency, (4) raw socket access. For CodeCrafters you can use C, C++, Go, Python, or Java — but understanding C makes you understand *why* Redis is fast.

### 3.2 Compilation Basics

compile.sh

```
# Compile a single file
gcc -o server server.c

# Compile with warnings (always do this!)
gcc -Wall -Wextra -o server server.c

# Compile C++ with C++17
g++ -std=c++17 -Wall -Wextra -o server server.cpp

# Compile multiple files
gcc -o server main.c server.c commands.c

# Debug build (includes debug symbols)
gcc -g -o server server.c
```

```
# Optimised release build
gcc -O2 -o server server.c
```

## 3.3 Makefile — Build Automation

Makefile

```
# Variables
CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -g
TARGET  = server
SRCS    = main.c server.c commands.c resp.c
OBJS    = $(SRCS:.c=.o)

# Default target
all: $(TARGET)

# Link
$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

# Compile each .c to .o
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# Clean build artifacts
clean:
    rm -f $(OBJS) $(TARGET)

# Run the server
run: $(TARGET)
    ./$(TARGET)

.PHONY: all clean run
```

TIP

Run `make` to build, `make clean` to remove binaries, `make run` to start the server. The `$(SRCS:.c=.o)` syntax replaces `.c` with `.o` for all source files automatically.

## 3.4 Key C Concepts for Redis

### 3.4.1 Pointers and Memory

pointers.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Heap allocation      you control the memory
    char *buf = malloc(1024);    // allocate 1024 bytes
    if (!buf) {
        perror("malloc failed");
        return 1;
    }

    memset(buf, 0, 1024);        // zero out the buffer
    strcpy(buf, "+PONG\r\n");    // write RESP response

    printf("%s", buf);
```

```
17
18      free(buf);                      // ALWAYS free what you malloc
19      buf = NULL;                     // prevent use-after-free
20      return 0;
21  }
```

### 3.4.2 Structs — Modelling Redis Data

structs.c

```
1   #include <time.h>
2
3   // A single Redis-style key-value entry
4   typedef struct {
5       char      *key;           // heap-allocated key string
6       char      *value;         // heap-allocated value string
7       time_t     expires_at;    // 0 means no expiry
8   } RedisEntry;
9
10  // A simple fixed-size hash table slot
11  typedef struct {
12      RedisEntry **entries;     // array of pointers
13      int          capacity;
14      int          size;
15  } RedisDB;
16
17  // Constructor
18  RedisDB *db_create(int capacity) {
19      RedisDB *db = malloc(sizeof(RedisDB));
20      db->entries  = calloc(capacity, sizeof(RedisEntry *));
21      db->capacity = capacity;
22      db->size     = 0;
23      return db;
24  }
```

### 3.4.3 Error Handling Pattern

error_handling.c

```
1   #include <errno.h>
2
3   // Redis convention: return -1 on error, 0 on success
4   int do_something(int fd) {
5       ssize_t n = write(fd, "+PONG\r\n", 7);
6       if (n == -1) {
7           perror("write");    // prints: "write: <errno message>"
8           return -1;
9       }
10      return 0;
11  }
```

**INTERVIEW Q&A**

**Q: What is a memory leak? How do you detect it?**

**A:** A memory leak occurs when heap memory is allocated with `malloc/calloc` but never `free`'d, causing the process to consume increasing memory over time.
Detect with: `valgrind -leak-check=full ./server`

**Q: What is a buffer overflow?**

**A:** Writing more bytes into a buffer than it can hold, corrupting adjacent memory. Real Redis carefully bounds every `read()` call. Use `strncpy` instead of `strcpy`, `snprintf` instead of `sprintf`.

# 4 Networking — TCP Sockets from Zero

**NOTE**

Redis listens on TCP port **6379** by default. Every Redis client opens a TCP connection and sends commands using the RESP protocol. You must understand how TCP sockets work in C.

## 4.1 The Socket Lifecycle

**Server side:** `socket()` → `setsockopt()` → `bind()` → `listen()` → `accept()` → `read/write()` → `close()`

**Client side:** `socket()` → `connect()` → `read/write()` → `close()`

## 4.2 Minimal TCP Server — Your First Milestone

server.c - Minimal TCP Server

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT    6379
#define BACKLOG 5
#define BUFSIZE 4096

int main() {
    /* 1. Create socket
       AF_INET  = IPv4
       SOCK_STREAM = TCP (reliable, ordered)
       0 = let OS pick protocol */
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) { perror("socket"); exit(1); }

    /* 2. Allow port reuse     prevents "Address already in use" */
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    /* 3. Bind to port */
    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;  // listen on all interfaces
    addr.sin_port        = htons(PORT); // htons = host-to-network byte order

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind"); exit(1);
    }

    /* 4. Start listening. BACKLOG = max pending connections */
    if (listen(server_fd, BACKLOG) == -1) {
        perror("listen"); exit(1);
    }
```

```c

    printf("Redis server listening on port %d\n", PORT);

    /* 5. Accept loop */
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t client_len = sizeof(client_addr);

        int client_fd = accept(server_fd,
                               (struct sockaddr *)&client_addr,
                               &client_len);
        if (client_fd == -1) { perror("accept"); continue; }

        printf("Client connected: %s\n",
               inet_ntoa(client_addr.sin_addr));

        /* 6. Read client data */
        char buf[BUFSIZE] = {0};
        ssize_t n = read(client_fd, buf, BUFSIZE - 1);
        if (n > 0) {
            printf("Received: %s\n", buf);
            /* 7. Send PONG response */
            write(client_fd, "+PONG\r\n", 7);
        }

        close(client_fd); /* 8. Close connection */
    }

    close(server_fd);
    return 0;
}
```

## 4.3 Testing Your Server

test_server.sh

```
# Terminal 1: Build and run
make && ./server

# Terminal 2: Test with redis-cli
redis-cli -p 6379 PING
# Expected: PONG

# Test with netcat (raw TCP)
echo -e "*1\r\n$4\r\nPING\r\n" | nc localhost 6379
# Expected: +PONG

# Test with curl (TCP)
curl -v telnet://localhost:6379
```

## 4.4 Handling Multiple Clients

### 4.4.1 Method 1: fork() — One Process Per Client

fork_server.c

```c
#include <sys/wait.h>

// Inside the accept loop:
int client_fd = accept(server_fd, ...);

pid_t pid = fork();
if (pid == 0) {
```

```
    // Child process: handle this client
    close(server_fd);       // child doesn't need listener
    handle_client(client_fd);
    close(client_fd);
    exit(0);
} else {
    // Parent process: go back to accept()
    close(client_fd);       // parent doesn't need this
    waitpid(-1, NULL, WNOHANG); // reap zombie children
}
```

### 4.4.2 Method 2: Threads — One Thread Per Client

threaded_server.c

```
#include <pthread.h>

void *handle_client_thread(void *arg) {
    int client_fd = *(int *)arg;
    free(arg);                    // we pass fd as heap pointer

    // ... handle client ...
    close(client_fd);
    return NULL;
}

// In accept loop:
int *client_fd_ptr = malloc(sizeof(int));
*client_fd_ptr = accept(server_fd, ...);

pthread_t tid;
pthread_create(&tid, NULL, handle_client_thread, client_fd_ptr);
pthread_detach(tid);   // auto-clean thread when done
```

compile_with_pthreads.sh

```
# Must link with -lpthread
gcc -Wall -g -o server server.c -lpthread
```

### 4.4.3 Method 3: I/O Multiplexing with select()/poll() (Production Style)

poll_server.c

```
#include <poll.h>

#define MAX_CLIENTS 1000

struct pollfd fds[MAX_CLIENTS + 1];

// Setup: index 0 = server_fd
fds[0].fd     = server_fd;
fds[0].events = POLLIN;
int nfds = 1;

while (1) {
    int ready = poll(fds, nfds, -1); // -1 = wait forever
    if (ready == -1) { perror("poll"); break; }

    // Check server socket for new connections
```

```
    if (fds[0].revents & POLLIN) {
        int client_fd = accept(server_fd, NULL, NULL);
        fds[nfds].fd     = client_fd;
        fds[nfds].events = POLLIN;
        nfds++;
    }

    // Check existing clients
    for (int i = 1; i < nfds; i++) {
        if (fds[i].revents & POLLIN) {
            char buf[4096] = {0};
            int n = read(fds[i].fd, buf, sizeof(buf) - 1);
            if (n <= 0) {
                close(fds[i].fd);
                fds[i] = fds[--nfds]; // compact array
            } else {
                process_command(fds[i].fd, buf, n);
            }
        }
    }
}
```

**INTERVIEW Q&A**

**Q: How does real Redis handle multiple clients?**

**A:** Real Redis uses a single-threaded **event loop** (similar to `epoll` on Linux / `kqueue` on macOS). It's I/O multiplexed — one thread handles thousands of connections. This avoids locking overhead since data structures are never accessed concurrently. Redis 6.0+ added multi-threaded I/O for reading/writing network data while keeping commands single-threaded.

**Q: What is epoll and why is it better than select?**

**A:** `select` has an O(n) scan across all watched fds on every call, and a limit of 1024 fds. `epoll` uses a kernel event queue — only ready fds are returned, making it O(1) per ready event.

# 5 RESP — Redis Serialization Protocol

**NOTE**

RESP (REdis Serialization Protocol) is the wire protocol all Redis clients and servers use. It is **human-readable**, simple to parse, and efficient. Every byte you read/write must follow RESP.

## 5.1 RESP2 Data Types

| First Byte | Type | Example |
|---|---|---|
| + | Simple String | `+OK\r\n` |
| - | Error | `-ERR unknown command\r\n` |
| : | Integer | `:42\r\n` |
| $ | Bulk String | `$6\r\nfoobar\r\n` |
| * | Array | `*2\r\n$3\r\nGET\r\n$3\r\nfoo\r\n` |

## 5.2 RESP Examples

RESP Wire Format Examples

```
# Client sends: PING
*1\r\n$4\r\nPING\r\n

# Server replies:
+PONG\r\n

# Client sends: SET foo bar
*3\r\n$3\r\nSET\r\n$3\r\nfoo\r\n$3\r\nbar\r\n

# Server replies:
+OK\r\n

# Client sends: GET foo
*2\r\n$3\r\nGET\r\n$3\r\nfoo\r\n

# Server replies (bulk string):
$3\r\nbar\r\n

# Null bulk string (key not found):
$-1\r\n

# Integer reply:
:1\r\n

# Error reply:
-ERR wrong number of arguments for 'get' command\r\n
```

## 5.3 RESP Parser Implementation

resp_parser.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ARGS 64
#define MAX_ARG_LEN 512

typedef struct {
    int    argc;
    char   argv[MAX_ARGS][MAX_ARG_LEN];
} Command;

// Parse a RESP array like: *2\r\n$3\r\nGET\r\n$3\r\nfoo\r\n
// Returns 0 on success, -1 on error
int parse_resp(const char *buf, Command *cmd) {
    cmd->argc = 0;
    const char *p = buf;

    // Expect '*' (array)
    if (*p != '*') return -1;
    p++;

    int num_args = atoi(p);
    while (*p && *p != '\r') p++;  // skip to \r\n
    p += 2;                        // skip \r\n

    for (int i = 0; i < num_args && i < MAX_ARGS; i++) {
        // Expect '$' (bulk string)
        if (*p != '$') return -1;
```

```
        p++;

        int len = atoi(p);
        while (*p && *p != '\r') p++;
        p += 2;  // skip \r\n

        // Copy the argument
        memcpy(cmd->argv[i], p, len);
        cmd->argv[i][len] = '\0';
        p += len + 2;  // skip data + \r\n

        cmd->argc++;
    }

    return 0;
}
```

## 5.4 Sending RESP Responses

resp_writer.c

```c
#include <unistd.h>
#include <stdio.h>

// Send: +OK\r\n
void send_ok(int fd) {
    write(fd, "+OK\r\n", 5);
}

// Send: +PONG\r\n
void send_pong(int fd) {
    write(fd, "+PONG\r\n", 7);
}

// Send: $-1\r\n  (null / not found)
void send_null(int fd) {
    write(fd, "$-1\r\n", 5);
}

// Send: -ERR message\r\n
void send_error(int fd, const char *msg) {
    char buf[256];
    int len = snprintf(buf, sizeof(buf), "-ERR %s\r\n", msg);
    write(fd, buf, len);
}

// Send: $N\r\nvalue\r\n  (bulk string)
void send_bulk(int fd, const char *value, int vlen) {
    char header[32];
    int hlen = snprintf(header, sizeof(header), "$%d\r\n", vlen);
    write(fd, header, hlen);
    write(fd, value, vlen);
    write(fd, "\r\n", 2);
}

// Send: :N\r\n  (integer)
void send_integer(int fd, long long n) {
    char buf[32];
    int len = snprintf(buf, sizeof(buf), ":%lld\r\n", n);
    write(fd, buf, len);
}
```

INTERVIEW Q&A

**Q: Why does Redis use RESP instead of JSON or Protobuf?**

**A:** RESP is designed specifically for client-server communication with minimal parsing overhead. It is human-readable (easy to debug with `nc`), easy to implement in any language, and efficient for both small commands and large bulk data. JSON adds overhead with escape processing; Protobuf requires schema compilation. RESP hits the sweet spot.

# 6 Core Redis Commands — Implementation

## 6.1 Command Dispatcher

commands.c - Dispatcher

```c
#include <string.h>

// Case-insensitive string compare
static int streq(const char *a, const char *b) {
    return strcasecmp(a, b) == 0;
}

void handle_command(int fd, Command *cmd, RedisDB *db) {
    if (cmd->argc == 0) {
        send_error(fd, "empty command");
        return;
    }

    const char *name = cmd->argv[0];

    if      (streq(name, "PING"))    cmd_ping(fd, cmd);
    else if (streq(name, "ECHO"))    cmd_echo(fd, cmd);
    else if (streq(name, "SET"))     cmd_set(fd, cmd, db);
    else if (streq(name, "GET"))     cmd_get(fd, cmd, db);
    else if (streq(name, "DEL"))     cmd_del(fd, cmd, db);
    else if (streq(name, "EXISTS"))  cmd_exists(fd, cmd, db);
    else if (streq(name, "EXPIRE"))  cmd_expire(fd, cmd, db);
    else if (streq(name, "TTL"))     cmd_ttl(fd, cmd, db);
    else if (streq(name, "KEYS"))    cmd_keys(fd, cmd, db);
    else if (streq(name, "HSET"))    cmd_hset(fd, cmd, db);
    else if (streq(name, "HGET"))    cmd_hget(fd, cmd, db);
    else if (streq(name, "LPUSH"))   cmd_lpush(fd, cmd, db);
    else if (streq(name, "RPUSH"))   cmd_rpush(fd, cmd, db);
    else if (streq(name, "LRANGE"))  cmd_lrange(fd, cmd, db);
    else {
        char err[128];
        snprintf(err, sizeof(err),
                 "unknown command `%s`", name);
        send_error(fd, err);
    }
}
```

## 6.2 PING and ECHO

ping_echo.c

```c
void cmd_ping(int fd, Command *cmd) {
    // PING with no args      PONG
    // PING "hello"      "hello" (bulk string)
    if (cmd->argc == 1) {
        send_pong(fd);
    } else {
        send_bulk(fd, cmd->argv[1], strlen(cmd->argv[1]));
```

```
    }
}

void cmd_echo(int fd, Command *cmd) {
    if (cmd->argc < 2) {
        send_error(fd, "wrong number of arguments for 'echo'");
        return;
    }
    send_bulk(fd, cmd->argv[1], strlen(cmd->argv[1]));
}
```

## 6.3 SET and GET with Expiry

set_get.c

```
#include <time.h>
#include <stdlib.h>

// SET key value [EX seconds] [PX milliseconds]
void cmd_set(int fd, Command *cmd, RedisDB *db) {
    if (cmd->argc < 3) {
        send_error(fd, "wrong number of arguments for 'set'");
        return;
    }

    const char *key    = cmd->argv[1];
    const char *value = cmd->argv[2];
    time_t expires_at = 0;

    // Parse optional EX / PX
    for (int i = 3; i < cmd->argc - 1; i++) {
        if (strcasecmp(cmd->argv[i], "EX") == 0) {
            int secs = atoi(cmd->argv[i + 1]);
            expires_at = time(NULL) + secs;
            i++;
        } else if (strcasecmp(cmd->argv[i], "PX") == 0) {
            long long ms = atoll(cmd->argv[i + 1]);
            expires_at = time(NULL) + (ms / 1000);
            i++;
        }
    }

    db_set(db, key, value, expires_at);
    send_ok(fd);
}

void cmd_get(int fd, Command *cmd, RedisDB *db) {
    if (cmd->argc < 2) {
        send_error(fd, "wrong number of arguments for 'get'");
        return;
    }

    const char *val = db_get(db, cmd->argv[1]);
    if (!val) {
        send_null(fd);
    } else {
        send_bulk(fd, val, strlen(val));
    }
}
```

### 6.4 DEL, EXISTS, TTL, EXPIRE

key_commands.c

```c
void cmd_del(int fd, Command *cmd, RedisDB *db) {
    int deleted = 0;
    for (int i = 1; i < cmd->argc; i++) {
        deleted += db_del(db, cmd->argv[i]);
    }
    send_integer(fd, deleted);
}

void cmd_exists(int fd, Command *cmd, RedisDB *db) {
    int count = 0;
    for (int i = 1; i < cmd->argc; i++) {
        if (db_exists(db, cmd->argv[i])) count++;
    }
    send_integer(fd, count);
}

void cmd_ttl(int fd, Command *cmd, RedisDB *db) {
    if (cmd->argc < 2) { send_error(fd, "wrong args"); return; }
    long long ttl = db_ttl(db, cmd->argv[1]);
    send_integer(fd, ttl);
    // -1 = key exists, no expiry
    // -2 = key does not exist
}

void cmd_expire(int fd, Command *cmd, RedisDB *db) {
    if (cmd->argc < 3) { send_error(fd, "wrong args"); return; }
    int secs = atoi(cmd->argv[2]);
    time_t expires_at = time(NULL) + secs;
    int found = db_set_expiry(db, cmd->argv[1], expires_at);
    send_integer(fd, found ? 1 : 0);
}
```

# 7 Data Structures Implementation

## 7.1 Hash Table (for String Keys)

hashtable.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define HT_INITIAL_CAP 16

typedef struct Entry {
    char         *key;
    char         *value;
    time_t        expires_at;   // 0 = never
    struct Entry *next;        // chaining for collisions
} Entry;

typedef struct {
    Entry **buckets;
    int     capacity;
    int     size;
} HashTable;

// DJB2 hash function
static unsigned long hash(const char *key, int cap) {
```

```
    unsigned long h = 5381;
    int c;
    while ((c = *key++))
        h = ((h << 5) + h) + c;
    return h % cap;
}

HashTable *ht_create(int cap) {
    HashTable *ht = malloc(sizeof(HashTable));
    ht->buckets   = calloc(cap, sizeof(Entry *));
    ht->capacity  = cap;
    ht->size      = 0;
    return ht;
}

void ht_set(HashTable *ht, const char *key,
            const char *val, time_t exp) {
    unsigned long idx = hash(key, ht->capacity);
    Entry *e = ht->buckets[idx];

    // Update existing key
    while (e) {
        if (strcmp(e->key, key) == 0) {
            free(e->value);
            e->value      = strdup(val);
            e->expires_at = exp;
            return;
        }
        e = e->next;
    }

    // Insert new entry
    Entry *ne   = malloc(sizeof(Entry));
    ne->key         = strdup(key);
    ne->value       = strdup(val);
    ne->expires_at  = exp;
    ne->next        = ht->buckets[idx];
    ht->buckets[idx] = ne;
    ht->size++;
}

const char *ht_get(HashTable *ht, const char *key) {
    unsigned long idx = hash(key, ht->capacity);
    Entry *e = ht->buckets[idx];
    while (e) {
        if (strcmp(e->key, key) == 0) {
            // Check expiry
            if (e->expires_at != 0 && time(NULL) > e->expires_at) {
                // Lazy expiration: key is expired
                return NULL;
            }
            return e->value;
        }
        e = e->next;
    }
    return NULL;  // Not found
}

int ht_del(HashTable *ht, const char *key) {
    unsigned long idx = hash(key, ht->capacity);
    Entry **pp = &ht->buckets[idx];
    while (*pp) {
        if (strcmp((*pp)->key, key) == 0) {
            Entry *dead = *pp;
```

```
            *pp = dead->next;
            free(dead->key);
            free(dead->value);
            free(dead);
            ht->size--;
            return 1;
        }
        pp = &(*pp)->next;
    }
    return 0;
}
```

**INTERVIEW Q&A**

**Q: How does Redis handle hash collisions?**

**A:** Redis uses separate chaining (linked lists at each bucket slot). When the load factor exceeds 1.0, Redis rehashes the table by doubling the capacity. Real Redis does **incremental rehashing** (rehash a few buckets per operation) to avoid blocking.

**Q: What is the average time complexity of GET/SET in Redis?**

**A:** O(1) average — hash table lookup. Worst case O(n) if all keys hash to same bucket (very unlikely with a good hash function).

## 7.2 Linked List (for LPUSH/RPUSH/LRANGE)

linkedlist.c

```c
typedef struct ListNode {
    char            *value;
    struct ListNode *prev;
    struct ListNode *next;
} ListNode;

typedef struct {
    ListNode *head;
    ListNode *tail;
    long long  length;
} RedisList;

RedisList *list_create() {
    RedisList *l = calloc(1, sizeof(RedisList));
    return l;
}

// LPUSH: insert at head
void list_lpush(RedisList *l, const char *val) {
    ListNode *n = malloc(sizeof(ListNode));
    n->value = strdup(val);
    n->prev  = NULL;
    n->next  = l->head;
    if (l->head) l->head->prev = n;
    l->head = n;
    if (!l->tail) l->tail = n;
    l->length++;
}

// RPUSH: insert at tail
void list_rpush(RedisList *l, const char *val) {
    ListNode *n = malloc(sizeof(ListNode));
    n->value = strdup(val);
    n->next  = NULL;
```

```
    n->prev  = l->tail;
    if (l->tail) l->tail->next = n;
    l->tail = n;
    if (!l->head) l->head = n;
    l->length++;
}

// LRANGE: return elements [start, stop]
// Negative indices: -1 = last, -2 = second to last
void list_lrange(int fd, RedisList *l, int start, int stop) {
    // Normalise negative indices
    if (start < 0) start = l->length + start;
    if (stop  < 0) stop  = l->length + stop;
    if (start < 0) start = 0;
    if (stop  >= l->length) stop = l->length - 1;

    if (start > stop) { write(fd, "*0\r\n", 4); return; }

    int count = stop - start + 1;
    char header[32];
    snprintf(header, sizeof(header), "*%d\r\n", count);
    write(fd, header, strlen(header));

    ListNode *cur = l->head;
    int idx = 0;
    while (cur && idx <= stop) {
        if (idx >= start)
            send_bulk(fd, cur->value, strlen(cur->value));
        cur = cur->next;
        idx++;
    }
}
```

## 7.3 Hash Maps (HSET/HGET)

hash_type.c

```
// A Redis Hash is a hash table inside a hash table
// Outer HT: key -> RedisObject (which contains inner HT)
// Inner HT: field -> value

typedef struct {
    char *field;
    char *value;
} HashField;

typedef struct {
    HashField **fields;
    int         capacity;
    int         size;
} RedisHash;

RedisHash *hash_create(int cap) {
    RedisHash *h = malloc(sizeof(RedisHash));
    h->fields    = calloc(cap, sizeof(HashField *));
    h->capacity  = cap;
    h->size      = 0;
    return h;
}

void hash_set(RedisHash *h, const char *field, const char *val) {
    // Use same DJB2 hash, store in h->fields array
    // (simplified     full impl uses chaining same as main HT)
```

```
    unsigned long idx = djb2(field) % h->capacity;
    h->fields[idx] = malloc(sizeof(HashField));
    h->fields[idx]->field = strdup(field);
    h->fields[idx]->value = strdup(val);
    h->size++;
}
```

# 8 Persistence — RDB and AOF

**NOTE**

Without persistence, all data is lost when the server restarts. Redis supports two mechanisms: **RDB** (point-in-time snapshots) and **AOF** (Append-Only File of commands). CodeCrafters requires RDB support.

## 8.1 RDB File Format

**RDB Binary Structure:**

`REDIS0011` | `FA` metadata | `FE` db-selector | `FB` resize-db | key-value pairs | `FF` checksum

rdb_reader.c - Reading RDB

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define RDB_MAGIC "REDIS"

// Read a length-encoded integer
// Returns the integer and advances buf pointer
uint64_t read_rdb_length(const unsigned char **buf) {
    unsigned char first = *(*buf)++;
    int encoding = (first & 0xC0) >> 6;

    if (encoding == 0) {
        // 6-bit length
        return first & 0x3F;
    } else if (encoding == 1) {
        // 14-bit length
        unsigned char second = *(*buf)++;
        return ((first & 0x3F) << 8) | second;
    } else if (encoding == 2) {
        // 32-bit big-endian length
        uint32_t len = 0;
        len |= (*(*buf)++) << 24;
        len |= (*(*buf)++) << 16;
        len |= (*(*buf)++) << 8;
        len |= (*(*buf)++);
        return len;
    }
    // encoding == 3: special encoding (integers stored as strings)
    return 0;
}

// Read a string (length-prefixed)
char *read_rdb_string(const unsigned char **buf) {
    uint64_t len = read_rdb_length(buf);
    char *str = malloc(len + 1);
    memcpy(str, *buf, len);
    str[len] = '\0';
```

```c
    *buf += len;
    return str;
}

// Load RDB file into our HashTable
int load_rdb(const char *path, HashTable *ht) {
    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    // Read entire file into buffer
    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    rewind(f);
    unsigned char *data = malloc(size);
    fread(data, 1, size, f);
    fclose(f);

    const unsigned char *p = data;

    // Verify magic + version
    if (memcmp(p, "REDIS", 5) != 0) { free(data); return -1; }
    p += 9; // skip "REDIS0011"

    // Parse sections
    while (*p != 0xFF) {
        if (*p == 0xFA) {
            // Auxiliary field
            p++;
            free(read_rdb_string(&p)); // key
            free(read_rdb_string(&p)); // value
        } else if (*p == 0xFE) {
            // Database selector
            p++;
            read_rdb_length(&p); // db number
        } else if (*p == 0xFB) {
            // Resize db hint
            p++;
            read_rdb_length(&p); // hash table size
            read_rdb_length(&p); // expiry hash table size
        } else if (*p == 0xFC) {
            // Key with millisecond expiry
            p++;
            uint64_t exp_ms = 0;
            memcpy(&exp_ms, p, 8); p += 8;
            p++; // value type (0 = string)
            char *key = read_rdb_string(&p);
            char *val = read_rdb_string(&p);
            time_t exp = exp_ms / 1000;
            ht_set(ht, key, val, exp);
            free(key); free(val);
        } else if (*p == 0xFD) {
            // Key with second expiry
            p++;
            uint32_t exp_s = 0;
            memcpy(&exp_s, p, 4); p += 4;
            p++; // value type
            char *key = read_rdb_string(&p);
            char *val = read_rdb_string(&p);
            ht_set(ht, key, val, (time_t)exp_s);
            free(key); free(val);
        } else {
            // No expiry      value type byte
            p++; // type (0 = string)
            char *key = read_rdb_string(&p);
```

```
            char *val = read_rdb_string(&p);
            ht_set(ht, key, val, 0);
            free(key); free(val);
        }
    }

    free(data);
    return 0;
}
```

## 8.2 Configuring RDB Path

config.c - Parse -dir and -dbfilename

```c
// Your server should support these CLI flags:
// ./server --port 6380 --dir /tmp --dbfilename dump.rdb

typedef struct {
    int  port;
    char dir[256];
    char dbfilename[256];
} Config;

Config *parse_args(int argc, char **argv) {
    Config *cfg = calloc(1, sizeof(Config));
    cfg->port = 6379;
    strcpy(cfg->dir, "/tmp");
    strcpy(cfg->dbfilename, "dump.rdb");

    for (int i = 1; i < argc - 1; i++) {
        if (strcmp(argv[i], "--port") == 0)
            cfg->port = atoi(argv[i + 1]);
        else if (strcmp(argv[i], "--dir") == 0)
            strncpy(cfg->dir, argv[i + 1], 255);
        else if (strcmp(argv[i], "--dbfilename") == 0)
            strncpy(cfg->dbfilename, argv[i + 1], 255);
    }
    return cfg;
}
```

INTERVIEW Q&A

**Q: What is the difference between RDB and AOF persistence?**

**A:**

- **RDB**: Binary snapshot of the dataset at a point in time. Compact, fast for restarts. Risk: data loss between snapshots. Good for backups.
- **AOF**: Logs every write command. Safer (less data loss). Larger file, slower restart. Can rewrite the AOF to compact it (`BGREWRITEAOF`).
- Real Redis recommends using **both** for maximum safety.

**Q: What is lazy expiration?**

**A:** Keys are not deleted when they expire. Instead, Redis checks expiry only when the key is accessed. This avoids scanning all keys constantly. Redis also runs a background job (active expiration) that samples random keys and removes expired ones.

# 9 Replication — Master/Replica

**NOTE**

Redis replication allows one master to have multiple replicas. Replicas get a full copy of data and then receive incremental command streams in real time.

## 9.1 Handshake Protocol

**Replica → Master handshake:**

1. Replica sends `PING` to master
2. Master replies `+PONG`
3. Replica sends `REPLCONF listening-port <port>`
4. Master replies `+OK`
5. Replica sends `REPLCONF capa psync2`
6. Master replies `+OK`
7. Replica sends `PSYNC ? -1` (first sync)
8. Master replies `+FULLRESYNC <replid> 0`
9. Master sends RDB file as bulk transfer
10. Master streams new write commands as they arrive

replication.c - Replica Handshake

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

// Connect replica to master and perform handshake
int replica_connect_to_master(const char *host, int port,
                              int my_port) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in master = {0};
    master.sin_family      = AF_INET;
    master.sin_port        = htons(port);
    inet_pton(AF_INET, host, &master.sin_addr);

    connect(fd, (struct sockaddr *)&master, sizeof(master));

    char buf[1024];

    // Step 1: PING
    write(fd, "*1\r\n$4\r\nPING\r\n", 14);
    read(fd, buf, sizeof(buf)); // read +PONG

    // Step 2: REPLCONF listening-port
    char msg[256];
    int len = snprintf(msg, sizeof(msg),
        "*3\r\n$8\r\nREPLCONF\r\n"
        "$14\r\nlistening-port\r\n$%d\r\n%d\r\n",
        (int)snprintf(NULL,0,"%d",my_port), my_port);
    write(fd, msg, len);
    read(fd, buf, sizeof(buf));

    // Step 3: REPLCONF capa psync2
    write(fd,
        "*3\r\n$8\r\nREPLCONF\r\n$4\r\ncapa\r\n$6\r\npsync2\r\n",
        44);
    read(fd, buf, sizeof(buf));

    // Step 4: PSYNC ? -1
    write(fd, "*3\r\n$5\r\nPSYNC\r\n$1\r\n?\r\n$2\r\n-1\r\n", 31);
```

```
    read(fd, buf, sizeof(buf));
    // buf now contains: +FULLRESYNC <replid> <offset>\r\n

    return fd; // keep this fd open to receive commands
}
```

### 9.2 INFO Command — Replication Status

info_replication.c

```
void cmd_info(int fd, Command *cmd, Config *cfg, int is_replica) {
    char info[1024];
    int len = snprintf(info, sizeof(info),
        "# Replication\r\n"
        "role:%s\r\n"
        "connected_slaves:0\r\n"
        "master_replid:8371b4fb1155b71f4a04d3e1bc3e18c4a990aeeb\r\n"
        "master_repl_offset:0\r\n"
        "repl_backlog_active:0\r\n",
        is_replica ? "slave" : "master"
    );

    send_bulk(fd, info, len);
}
```

**INTERVIEW Q&A**

**Q: Is Redis replication synchronous or asynchronous?**

**A:** Redis replication is **asynchronous by default**. The master acknowledges writes to clients before replicas confirm receipt. This gives high performance but means a replica could be slightly behind (replication lag). Using `WAIT` command can force semi-synchronous behaviour.

**Q: What is a replication ID?**

**A:** A 40-character hex string that uniquely identifies a replication stream. Each time a master restarts or a replica is promoted, a new ID is generated. Replicas use it to know if they can do a partial resync (`PSYNC`) or need a full resync.

## 10 Complete Build: Step-by-Step

**WARNING**

Follow these steps **in order**. Each step produces a testable milestone. Don't skip ahead — each stage builds on the last.

### 10.1 Project Structure

Project Structure

```
my-redis/
          Makefile
          src/
                main.c          # Entry point, config parsing
                server.c        # Socket setup, accept loop
                commands.c      # Command dispatcher
                resp.c          # RESP parser and writer
                hashtable.c     # String key-value store
                list.c          # Linked list for lists
                hash.c          # Hash type (HSET/HGET)
                rdb.c           # RDB persistence
```

```
        replication.c    # Replication logic
        config.c         # Config struct and CLI parsing
    include/
        server.h
        commands.h
        resp.h
        hashtable.h
        rdb.h
    tests/
        test_resp.c
```

## 10.2 Stage 1 — TCP Echo Server

**Stage 1: Bind to Port 6379 and Accept Connections**

**Goal:** Server starts, accepts a TCP connection, reads data, closes.

**Test:** `redis-cli -p 6379 PING` → doesn't crash

**Files:** `main.c`, `server.c`

## 10.3 Stage 2 — RESP Parser + PING

**Stage 2: Parse RESP and Handle PING**

**Goal:** Parse `*1\r\n$4\r\nPING\r\n`, respond `+PONG\r\n`

**Test:** `redis-cli PING` → `PONG`

**Files:** `resp.c`, `commands.c`

## 10.4 Stage 3 — Concurrent Clients

**Stage 3: Handle Multiple Clients**

**Goal:** Two clients can connect simultaneously and both receive responses.

**Approach:** Use `fork()` or `pthread` (simplest to start)

**Test:** Run two `redis-cli` terminals simultaneously

## 10.5 Stage 4 — SET/GET

**Stage 4: In-Memory Key-Value Store**

**Goal:** Implement hash table; handle SET, GET, DEL, EXISTS

**Test:**

```
redis-cli SET foo bar    # OK
redis-cli GET foo        # bar
redis-cli GET baz        # (nil)
redis-cli DEL foo        # 1
```

## 10.6 Stage 5 — Expiry

**Stage 5: Key Expiry (EX, PX, EXPIRE, TTL)**

**Goal:** Keys expire after the given duration; TTL returns remaining time

**Test:**

```
redis-cli SET foo bar EX 5
redis-cli TTL foo            # 4 or 5
sleep 6
```

```
redis-cli GET foo            # (nil)
```

### 10.7 Stage 6 — RDB Persistence

**Stage 6: Load Existing RDB on Startup**

**Goal:** On startup, load `dump.rdb` from `-dir` into the hash table

**Test:**

```
# Run real Redis, set some keys, stop it
# Copy dump.rdb to /tmp/
# Start your server: ./server --dir /tmp --dbfilename dump.rdb
redis-cli GET key_you_set_before    # should return the value
```

### 10.8 Stage 7 — Replication

**Stage 7: Master/Replica Replication**

**Goal:** Replica connects to master, completes handshake, receives full RDB, then receives command stream

**Test:**

```
# Terminal 1: Start master
./server --port 6379

# Terminal 2: Start replica
./server --port 6380 --replicaof localhost 6379

# Terminal 3: Set on master, get on replica
redis-cli -p 6379 SET foo bar
redis-cli -p 6380 GET foo     # bar
```

## 11 Debugging and Testing

### 11.1 Valgrind — Catch Memory Bugs

valgrind.sh

```
# Run your server under valgrind
valgrind --leak-check=full \
         --track-origins=yes \
         --show-reachable=yes \
         ./server

# In another terminal, run some commands, then Ctrl+C server
# Valgrind prints: "definitely lost: 0 bytes in 0 blocks" = good
```

### 11.2 GDB — Step Through Code

gdb.sh

```
# Build with debug symbols: gcc -g ...
gdb ./server

# Inside GDB:
(gdb) break server.c:45        # set breakpoint at line 45
(gdb) run                      # start program
(gdb) next                     # step over
(gdb) step                     # step into
```

```
(gdb) print buf                # print variable
(gdb) bt                       # backtrace (stack frames)
(gdb) continue                 # resume execution
(gdb) quit
```

### 11.3 Testing with redis-cli and nc

test_commands.sh

```
# Test PING
redis-cli -p 6379 PING

# Test SET/GET/DEL
redis-cli SET name Redis
redis-cli GET name
redis-cli DEL name

# Test expiry
redis-cli SET temp value EX 2
redis-cli TTL temp
sleep 3
redis-cli EXISTS temp     # 0

# Test hash
redis-cli HSET user:1 name Alice age 30
redis-cli HGET user:1 name

# Test list
redis-cli RPUSH mylist a b c
redis-cli LRANGE mylist 0 -1

# Send raw RESP
printf "*1\r\n$4\r\nPING\r\n" | nc localhost 6379
```

## 12 Interview Preparation — Complete Q&A

INTERVIEW Q&A

**Q: What is Redis and what makes it fast?**

**A:** Redis (Remote Dictionary Server) is an open-source, in-memory data structure store. It is fast because:

- All data lives in RAM (no disk I/O on reads)
- Single-threaded command execution (no lock contention)
- Event-loop I/O multiplexing (epoll/kqueue)
- Highly optimised C data structures
- Simple protocol (RESP) with minimal overhead

INTERVIEW Q&A

**Q: What data types does Redis support?**

**A:**

- **String**: Binary-safe, up to 512 MB. Underlying: SDS (Simple Dynamic String)
- **List**: Doubly linked list or ziplist. O(1) push/pop from head/tail
- **Hash**: Field-value pairs. Small hashes use ziplist, large ones use hashtable
- **Set**: Unique, unordered elements. Uses intset or hashtable
- **Sorted Set**: Elements with float scores. Uses skiplist + hashtable
- **Stream**: Append-only log (Redis 5+)

- **HyperLogLog**: Probabilistic cardinality counting
- **Geospatial**: Sorted set encoding lat/long

**INTERVIEW Q&A**

**Q: Explain Redis key expiration mechanisms.**

**A:** Redis uses two strategies:

- **Lazy expiration**: Check if a key is expired only when accessed. No background work.
- **Active expiration**: A background job runs 10 times per second, samples 20 random keys from each database. If more than 25% are expired, repeat. This bounds the number of stale keys at any time.

**INTERVIEW Q&A**

**Q: What is the difference between SAVE and BGSAVE?**

**A:**

- `SAVE`: Blocks the server while writing RDB. No requests are served.
- `BGSAVE`: Forks a child process. Child writes RDB while parent continues serving requests. Uses copy-on-write — parent and child share pages until parent modifies one.

**INTERVIEW Q&A**

**Q: How does Redis handle atomic operations?**

**A:** Since Redis is single-threaded for command execution, every command is atomic by definition — no two commands run concurrently. For compound atomicity, Redis provides:

- **MULTI/EXEC**: Queues commands, executes as a block
- **WATCH**: Optimistic locking — if a key changes between WATCH and EXEC, transaction aborts
- **Lua scripts**: Evaluated atomically with `EVAL`

**INTERVIEW Q&A**

**Q: What is a Redis Cluster?**

**A:** Redis Cluster is a built-in sharding solution that partitions data across multiple masters using **hash slots** (0–16383). Each key maps to a slot via `CRC16(key) % 16384`. Each master owns a subset of slots. Cluster provides horizontal scaling and automatic failover.

**INTERVIEW Q&A**

**Q: What is Sentinel?**

**A:** Redis Sentinel monitors masters and replicas, providing automatic failover (promotes a replica to master if the master fails) and service discovery. It runs as separate processes and requires a quorum to agree on a failure before acting.

**INTERVIEW Q&A**

**Q: Name some common Redis use cases.**

**A:**

- **Cache**: Store computed results to reduce database load
- **Session store**: Fast user session reads
- **Rate limiting**: INCR + EXPIRE per user per time window
- **Pub/Sub messaging**: Channels and subscribers

- **Leaderboard**: Sorted sets for score-ranked boards
- **Distributed locks**: SETNX (Set if Not eXists) pattern
- **Job queues**: List as queue with LPUSH/BRPOP

INTERVIEW Q&A

**Q: What is an SDS (Simple Dynamic String)?**

**A:** Real Redis does not use C's native `char *` for strings. Instead it uses SDS, a custom string structure that stores the string length, free space, and the bytes inline. Benefits: O(1) `strlen()`, binary-safe (can store null bytes), less risk of buffer overflows.

INTERVIEW Q&A

**Q: Explain the copy-on-write mechanism used in BGSAVE.**

**A:** When Redis calls `fork()`, the OS creates a child process that shares all memory pages with the parent via copy-on-write. Pages are only copied when either process writes to them. The child (writing RDB) reads pages without modifying them; only the parent (serving new writes) causes copies. This makes BGSAVE memory-efficient for read-heavy workloads.

## 13 Quick Reference Cheat Sheet

**Essential Redis Commands:**

| Command | Syntax | Returns |
|---|---|---|
| PING | `PING [msg]` | PONG / msg |
| SET | `SET key val [EX sec]` | OK |
| GET | `GET key` | value / nil |
| DEL | `DEL key [key ...]` | integer (deleted count) |
| EXISTS | `EXISTS key` | 1 / 0 |
| EXPIRE | `EXPIRE key sec` | 1 / 0 |
| TTL | `TTL key` | seconds / -1 / -2 |
| KEYS | `KEYS pattern` | list of keys |
| TYPE | `TYPE key` | string/list/hash/set |
| HSET | `HSET key field val` | integer |
| HGET | `HGET key field` | value / nil |
| LPUSH | `LPUSH key val [val ...]` | list length |
| RPUSH | `RPUSH key val [val ...]` | list length |
| LRANGE | `LRANGE key 0 -1` | list elements |
| INCR | `INCR key` | new integer value |
| INFO | `INFO [section]` | server info |

**RESP Type Cheat Sheet:**

| Send/Receive | Wire Format | C Function |
|---|---|---|
| OK | `+OK\r\n` | `send_ok(fd)` |
| PONG | `+PONG\r\n` | `send_pong(fd)` |
| Null | `$-1\r\n` | `send_null(fd)` |
| Error | `-ERR msg\r\n` | `send_error(fd, msg)` |
| Bulk | `$N\r\ndata\r\n` | `send_bulk(fd, data, n)` |
| Integer | `:N\r\n` | `send_integer(fd, n)` |

**Git Quick Reference:**

| Command | Action |
|---|---|
| `git init` | Start tracking a folder |
| `git clone <url>` | Copy remote repo locally |
| `git status` | See what changed |
| `git add .` | Stage all changes |
| `git commit -m "msg"` | Save snapshot |
| `git push origin main` | Upload to GitHub |
| `git pull` | Download latest changes |
| `git log -oneline` | See commit history |
| `git checkout -b feat` | Create + switch branch |
| `git merge feat` | Merge branch into current |

## You Are Ready to Build Redis.

Follow the 7 stages in order. Test after every stage.
When something breaks, use valgrind and gdb.
Every line in this document is something you need.

**Good luck. Build something real.**